

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10	Course Title:	Software Engineering
CO Assessed:	CO2	Assessment:	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	Total Marks:	25
CO2 Statement:	Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills		
Student Name:	Surendar M G	Reg. No.:	192521374
Date:	04-08-2026	Duration:	60 minutes

Code of Conduct

I Surendar M G certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the Student: Surendar M G

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.

Selection of Software Architecture	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design	Produces a clear, complete, and wellstructured architecture diagram with all major components,	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and	Diagram is incomplete, unclear, or technically incorrect.
	interfaces, and interactions accurately represented.		lacks sufficient detail.	
Component Interaction and Quality Attribute Analysis	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a wellorganized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

Criteria	Mark
System Requirement Analysis	/5
Selection of Software Architecture	/5
Architecture Diagram and Component Design	/5
Component Interaction and Quality Attribute Analysis	/5
Architectural Justification and Technical Presentation	/5

	TOTAL	/5
--	-------	----

Architecture Design Report

Smart Forest Fire Detection and Environmental Monitoring System

Annexure A — Architecture Design Assignment

Course: Software Engineering

1. System Requirements Analysis

1.1 Problem Context and System Objectives

Conventional forest fire monitoring relies on manual observation and satellite passes, which typically detect fires only after significant spread has occurred, resulting in severe ecological and economic damage. The proposed Smart Forest Fire Detection and Environmental Monitoring System addresses this gap by continuously collecting environmental data through IoT sensors, drones, and satellite feeds, and applying AI-based analysis to detect and predict fire risk early, so that alerts reach emergency responders before a fire spreads uncontrollably.

The system objectives, and the primary architectural component responsible for each, are summarized below:

System Objective	Primary Responsible Component
Monitor forest environmental conditions continuously	Environmental Monitoring Service
Detect forest fires at an early stage	Fire Detection Service
Predict fire risk using AI	AI Fire Risk Prediction Service
Integrate drone and satellite monitoring	Drone/Satellite Image Analysis Service
Generate emergency alerts	Alert & Notification Service
Support firefighting operations	Emergency Response Coordination Service
Produce environmental analytics reports	Analytics & Reporting Service
Improve forest conservation	System-wide data-driven outcome

1.2 Functional Requirements

- Continuously collect temperature, humidity, smoke concentration, and wind speed data from distributed IoT sensor nodes.
- Ingest and process periodic drone-captured thermal/visual imagery and satellite feeds.
- Run AI/ML models to compute a real-time fire risk score per forest zone and detect active fire signatures.
- Trigger and dispatch immediate multi-channel alerts (SMS, push notification, dashboard, siren integration) when risk thresholds are breached.
- Provide a live GIS-based dashboard showing sensor status, risk heat-maps, and active incidents.
- Allow emergency response agencies to receive alerts programmatically via API and coordinate firefighting resources.
- Generate periodic environmental and incident analytics reports for forest departments and researchers.
- Support historical data query, sensor device management, and system configuration by administrators.

1.3 Non-Functional Requirements and Quality Attributes

Because the platform is safety-critical — a missed or delayed alert can allow a wildfire to spread unchecked — the following non-functional requirements directly shaped the architectural style selected in Section 2:

Quality Attribute	Requirement Description
-------------------	-------------------------

Scalability	The platform must scale horizontally to support thousands of sensors, multiple drones, and satellite feeds across large, geographically dispersed forest regions without
Quality Attribute	Requirement Description
	degradation.
Performance	Fire-risk detection and alert generation must complete within seconds of anomalous sensor readings to enable rapid response (near real-time processing).
Reliability	The system must reliably deliver every critical alert; sensor data loss or a single service failure must not silently suppress a fire warning.
Availability	The monitoring and alerting path must remain available 24x7, including during partial network outages in remote forest areas (offline buffering at the edge).
Security	Sensor data, drone/satellite feeds, and emergency communications must be protected against tampering, spoofing, and unauthorized access (data integrity is safety-critical).
Maintainability	Individual capabilities (e.g., AI model updates, new sensor types) must be upgradable independently without redeploying the entire system.
Interoperability	The system must integrate with third-party weather APIs, government disastermanagement systems, and satellite data providers via standard protocols.
Usability	Dashboards and mobile alerts must be simple enough for field rangers and emergency responders to act on quickly under stress.

2. Selected Software Architecture

2.1 Architectural Style: Hybrid (Event-Driven Architecture + Microservices, with a Layered Presentation Tier)

A single classical style does not, on its own, satisfy every requirement of this system. Instead, a Hybrid Architecture is proposed, combining three complementary styles at different points in the system:

Event-Driven Architecture (EDA) as the backbone — IoT sensor readings, drone captures, and satellite updates are naturally continuous streams of events. An event bus (e.g., Apache Kafka) decouples data producers from the many consumers (detection, prediction, alerting, storage, analytics) that must react to the same data independently and in near real time.

Microservices for the application layer — Fire detection, AI risk prediction, image analysis, alerting, and reporting are distinct capabilities with different scaling needs, technology stacks (e.g., Python for ML vs. Node.js for APIs), and release cadences. Packaging them as independently deployable microservices supports fault isolation and rapid iteration.

Layered / Client-Server structure for the presentation tier — The dashboard, mobile app, and public portal consume backend services through a well-defined API Gateway layer, giving a simple, familiar layered separation between presentation, application logic, and data.

2.2 Justification

- Real-time responsiveness: Fire risk can escalate within minutes, so an event-driven core enables sub-second propagation of sensor anomalies to detection and alerting services, which a traditional request-response monolith cannot match at this scale.

- Independent scalability: Sensor telemetry volume, drone image processing, and AI inference have very different, unpredictable load profiles; microservices let each scale independently rather than over-provisioning a single monolith.
- Fault isolation and safety: If the AI risk-prediction service fails or is being updated, the simpler rule-based Fire Detection Service and Alert Service must keep operating — a property naturally supported by loosely coupled microservices, but very difficult to guarantee in a monolith.
- Heterogeneous integration: The system must integrate IoT protocols (MQTT/LoRaWAN), computer-vision pipelines, satellite data feeds, and third-party emergency-agency systems. An API Gateway plus event bus provides a consistent integration surface for this diversity.
- Future extensibility: New sensor types, additional AI models, or new consumer applications (e.g., a citizen-reporting app) can be added as new event subscribers or new microservices without redesigning the whole system.

3. Software Architecture Diagram

The diagram below illustrates the full system structure — from field data sources through event messaging, microservices, persistence, and presentation, along with the cross-cutting concerns applied across all layers.

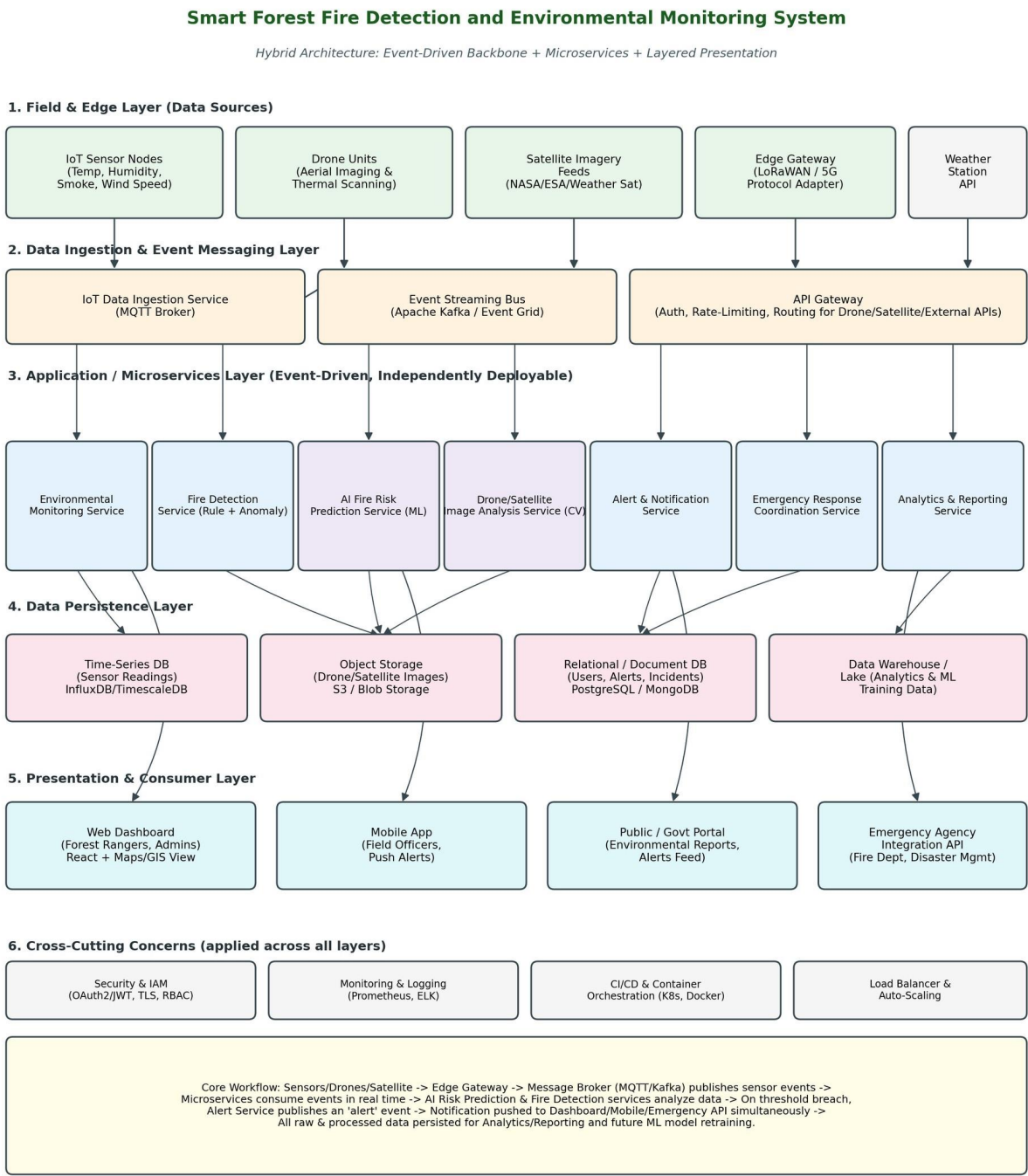


Figure 1: Hybrid Event-Driven Microservices Architecture for the Smart Forest Fire Detection System

4. Components and Their Interactions

4.1 Component Responsibilities

Component	Purpose / Responsibility
IoT Sensor Nodes	Field devices measuring temperature, humidity, smoke concentration, and wind speed; transmit readings periodically over LoRaWAN/5G to the Edge Gateway.
Drone Units	Capture aerial thermal and visual imagery of high-risk zones on scheduled or triggered flights; upload data via the API Gateway.
Satellite Imagery Feed	External data source (e.g., weather/fire satellites) providing wide-area thermal anomaly and vegetation-dryness data.
Edge Gateway	Aggregates nearby sensor traffic, performs local buffering during connectivity loss, and forwards data upstream, reducing bandwidth and improving resilience.
IoT Data Ingestion Service (MQTT Broker)	Lightweight publish/subscribe endpoint that receives high-frequency sensor telemetry and republishes it as structured events.
Event Streaming Bus (Kafka/Event Grid)	Central nervous system of the architecture; decouples producers (sensors, drones) from consumers (microservices), enabling asynchronous, real-time, fan-out processing.
API Gateway	Single secured entry point for external and internal API calls (drone uploads, satellite integration, dashboard/mobile clients); handles authentication, throttling, and routing.
Environmental Monitoring Service	Continuously aggregates and normalizes live sensor readings; exposes current environmental state per zone.
Fire Detection Service	Applies rule-based thresholds and statistical anomaly detection to raw telemetry to flag possible fire signatures instantly.
AI Fire Risk Prediction Service	Machine learning models (trained on historical weather, vegetation, and incident data) compute a continuous fire-risk score and short-term forecast per zone.
Drone/Satellite Image Analysis Service	Computer-vision models process aerial and satellite imagery to detect smoke plumes, hotspots, and burn-scar progression.
Alert & Notification Service	Consumes detection/prediction events and dispatches multi-channel alerts (push, SMS, dashboard, siren API) based on severity.
Emergency Response Coordination Service	Manages incident lifecycle, resource assignment, and two-way communication with fire departments and disaster-management agencies.
Analytics & Reporting Service	Runs batch/scheduled analytics over historical data to produce environmental trend reports and supports ML retraining datasets.
Time-Series Database	Optimized store (InfluxDB/TimescaleDB) for high-volume, timestamped sensor readings enabling fast range queries.

Object Storage	Stores raw drone photos, thermal images, and satellite tiles cost-effectively at scale (S3/Blob Storage).
Relational/Document Database	Stores structured operational data: users, device registry, alerts, and incident records.
Data Warehouse / Lake	Consolidated historical store used for analytics dashboards and training future AI models.
Component	Purpose / Responsibility
Web Dashboard	GIS-based operational console for forest rangers and administrators to visualize live risk heat-maps and manage devices.
Mobile App	Field application for rangers/firefighters to receive push alerts and report ground observations.
Public/Government Portal	Publishes environmental reports and public safety alerts for transparency and community awareness.
Emergency Agency Integration API	Machine-to-machine interface allowing fire departments' own systems to consume alerts and incident data automatically.

4.2 Communication Mechanisms

- **Sensor-to-Edge:** Low-power wireless protocols (LoRaWAN) or 5G carry raw telemetry from IoT nodes to the Edge Gateway.
- **Edge/Drone/Satellite-to-Cloud:** The Edge Gateway and drone units publish data over MQTT to the IoT Data Ingestion Service; satellite and other external providers connect via REST/HTTPS through the API Gateway.
- **Service-to-Service (asynchronous):** All ingested data is published as events onto the Kafka event bus; the Fire Detection, AI Risk Prediction, and Image Analysis services subscribe independently and process events in parallel.
- **Service-to-Service (synchronous):** Where an immediate response is required (e.g., a dashboard requesting the current risk score), services expose REST/gRPC APIs behind the API Gateway.
- **Service-to-Data:** Each microservice owns and accesses only its relevant datastore (time-series DB, object storage, relational DB, or warehouse), following the database-per-service pattern to keep services loosely coupled.
- **Backend-to-Consumers:** The Alert Service pushes notifications via webhooks/push-notification services (FCM/APNs) to the mobile app and via WebSockets/SSE to the live dashboard; the Emergency Agency Integration API exposes REST endpoints and webhooks for fire departments.

4.3 Overall Workflow

1. IoT sensors continuously stream temperature, humidity, smoke, and wind data to the Edge Gateway, which forwards it to the IoT Data Ingestion Service and onward as events on the Kafka bus.
2. Drones perform scheduled or on-demand flights over high-risk zones, and satellite feeds are polled periodically; both are ingested through the API Gateway and published as image/analysis events.
3. The Fire Detection Service applies real-time rule and anomaly checks, while the AI Fire Risk Prediction Service and Image Analysis Service run machine-learning and computer-vision models on the same event stream in parallel.
4. When a fire signature or a high risk score is detected, an 'alert' event is published; the Alert & Notification Service immediately fans this out to the Web Dashboard, Mobile App, Public Portal, and the Emergency Agency Integration API simultaneously.

5. The Emergency Response Coordination Service tracks the incident lifecycle and resource assignment as firefighting agencies act on the alert.
6. All raw and processed data is persisted in the appropriate datastore; the Analytics & Reporting Service later mines this historical data to produce environmental reports and retraining datasets for the AI models, closing the feedback loop for continuous improvement.

5. Quality Attributes Evaluation

The table below evaluates how the proposed hybrid architecture specifically addresses each required quality attribute.

Attribute	How the Architecture Addresses It
Scalability	Each microservice scales independently via container orchestration (Kubernetes) and autoscaling groups; the Kafka event bus absorbs bursts of sensor/drone traffic without overloading downstream services, allowing the platform to grow from a single forest reserve to national coverage.
Security	TLS-encrypted transport, OAuth2/JWT-based authentication and role-based access control at the API Gateway, sensor-device identity certificates, and encrypted storage protect against spoofed readings and unauthorized access to safety-critical alerts.
Performance	Event-driven, asynchronous processing lets the Fire Detection and AI Risk Prediction services react to sensor events within seconds; the time-series database is optimized for high write-throughput and low-latency range queries needed for real-time dashboards.
Reliability	The Edge Gateway buffers data during connectivity loss; Kafka provides durable, replayable event logs so no sensor reading or alert event is lost even if a downstream microservice temporarily fails; retry and dead-letter queues handle transient errors.
Maintainability	Loose coupling between microservices, each with a bounded responsibility, allows independent development, testing, deployment, and scaling — for example, upgrading the AI model does not require touching the alerting or dashboard code.
Availability	Redundant service instances behind load balancers, multi-zone database replication, and graceful degradation (cached last-known state) keep the monitoring and alerting path operational close to 24x7, even during partial infrastructure failures.

6. Justification of Architectural Decisions and Trade-offs

6.1 Rationale Summary

The hybrid event-driven microservices architecture was chosen over simpler alternatives (a single layered monolith or a pure client-server design) because the problem domain is inherently characterized by continuous, high-volume, multi-source data streams (sensors, drones, satellites) that must be processed by multiple independent analytical capabilities within tight time constraints where safety is at stake. Only an asynchronous, loosely coupled, independently scalable design can meet the combined demands of real-time responsiveness, fault isolation, and heterogeneous system integration identified in Section 1.

6.2 Trade-offs Considered

Decision	Choice Made	Trade-off Accepted
----------	-------------	--------------------

Microservices vs. Monolith	Chosen: Microservices	Higher operational complexity (deployment, orchestration, distributed tracing) is accepted in exchange for independent scalability and fault isolation, which are essential since a slow AI model must never delay basic alerting.
Event-Driven (async) vs. Request-Response (sync)	Chosen: Event-Driven backbone, with sync APIs at the edges	Slight increase in end-to-end latency accounting and eventual consistency is accepted in exchange for high throughput, loose coupling, and natural support for realtime fan-out to multiple consumers (dashboard, mobile, emergency API) simultaneously.
Decision	Choice Made	Trade-off Accepted
Polyglot data stores vs. single database	Chosen: Polyglot persistence (time-series, object, relational, warehouse)	Added integration/maintenance overhead is accepted because each data shape (telemetry, imagery, operational records, analytics) has very different access patterns that a single database would serve poorly.
Edge buffering vs. alwaysonline sensors	Chosen: Edge Gateway with local buffering	Extra hardware/software at the edge is accepted to guarantee no data loss in remote, connectivity-poor forest regions.

6.3 Support for Future Enhancements and Long-Term Maintainability

- New data sources (e.g., additional sensor types, new drone models, next-generation satellites) can be onboarded by adding new producers to the event bus without modifying existing consumers.
- New AI/ML models can be deployed as new versions of the AI Fire Risk Prediction Service and A/B tested by subscribing to the same event stream, without disrupting the rule-based Fire Detection Service that provides a safety baseline.
- New consumer applications (e.g., a citizen-reporting mobile app, or a national forest-management portal) can be added simply as new subscribers to alert and analytics events.
- Because each microservice is independently deployable and owns its own data, teams can maintain, test, and release updates to individual capabilities (e.g., improving the image-analysis model) without a full-system redeployment, supporting long-term maintainability as the platform grows to cover more forest regions.
- Standardized REST/event contracts at the API Gateway and event bus make it straightforward to integrate with external systems such as national disaster-management platforms, insurance risk databases, or research institutions in the future.

7. Conclusion

The Smart Forest Fire Detection and Environmental Monitoring System requires an architecture capable of processing continuous, high-volume sensor, drone, and satellite data, running real-time AI analysis, and reliably delivering time-critical alerts to emergency responders. The proposed Hybrid Event-Driven Microservices Architecture, layered behind a secured API Gateway and consumed through Web, Mobile, and Public-Portal presentation channels, satisfies the identified functional and non-functional requirements while remaining scalable, reliable, secure, and maintainable — providing a solid foundation for early wildfire detection and long-term, data-driven forest conservation.